🌐 Ejercicios: Auditoría básica de páginas web

Objetivo: Analizar vulnerabilidades comunes en una página web usando herramientas incluidas en Kali Linux.

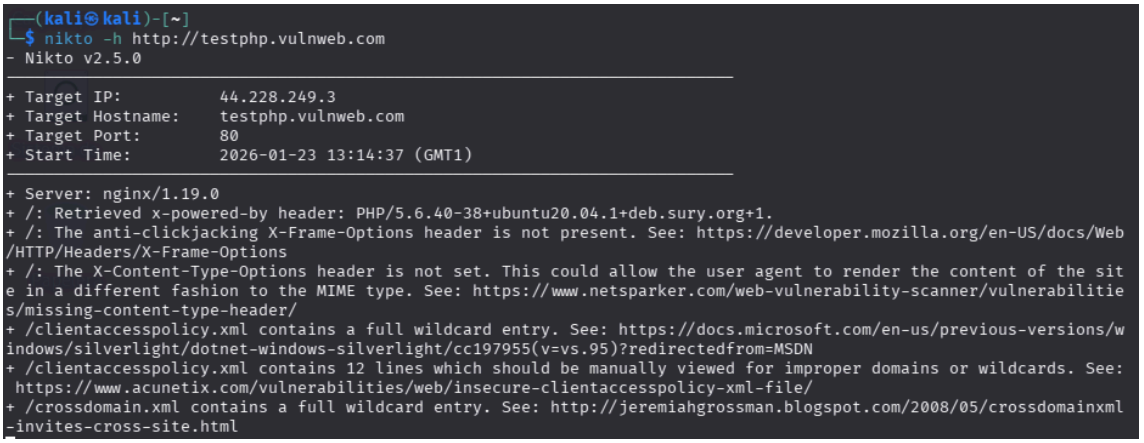
Material necesario

- Kali Linux.
- Una página web de pruebas (puede ser una aplicación vulnerable en local, como DVWA o WebGoat, o un sitio autorizado para prácticas, como **testphp.vulnweb.com**).

Pasos

1. Haz un escaneo con **Nikto** para identificar configuraciones inseguras y vulnerabilidades:

   nikto -h http://testphp.vulnweb.com



2. Analiza con **WhatWeb** qué tecnologías usa el sitio (servidor, CMS, etc.):

   whatweb http://testphp.vulnweb.com

```
┌──(kali㉿kali)-[~]
└─$ whatweb http://testphp.vulweb.com
http://testphp.vulweb.com [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[openresty/1.27.1.2], IP[172.104.251.198
], OpenResty[1.27.1.2], Script, Strict-Transport-Security[max-age=0; includeSubDomains; preload], Title[vulweb.com]
```

3. Opcional: Haz un escaneo con **sqlmap** para comprobar si un formulario es vulnerable a inyección SQL:

   sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" –batch

   si queremos listar las bases de datos sería:

   sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" --dbs

```
┌──(kali㉿kali)-[~]
└─$ sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" --dbs
        ___
       __H__
 ___ ___[']_____ ___ ___  {1.10#stable}
|_ -| . ["]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end
user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are
not responsible for any misuse or damage caused by this program

[*] starting @ 13:22:25 /2026-01-23/

[13:22:25] [INFO] testing connection to the target URL
[13:22:26] [INFO] checking if the target is protected by some kind of WAF/IPS
[13:22:26] [INFO] testing if the target URL content is stable
[13:22:26] [INFO] target URL content is stable
[13:22:26] [INFO] testing if GET parameter 'artist' is dynamic
[13:22:27] [INFO] GET parameter 'artist' appears to be dynamic
[13:22:27] [INFO] heuristic (basic) test shows that GET parameter 'artist' might be injectable (possible DBMS: 'MyS
QL')
[13:22:27] [INFO] heuristic (XSS) test shows that GET parameter 'artist' might be vulnerable to cross-site scriptin
g (XSS) attacks
[13:22:27] [INFO] testing for SQL injection on GET parameter 'artist'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n]
```

Resultado esperado

- Informe de posibles vulnerabilidades (cabeceras inseguras, directorios ocultos, configuraciones por defecto).
- Identificación de tecnologías utilizadas en la web (PHP, Apache, etc.).
- Detección (si la hay) de parámetros vulnerables a inyección SQL.